

# 01

GESTIÓN DE PROYECTOS ÁGILES · UBP

## Gestión de Proyectos y Fundamentos de Agilidad

*Del proyecto como esfuerzo temporal a la agilidad como respuesta profesional frente a la incertidumbre.*

---

### EN ESTE MÓDULO

1. Proyecto y gestión
2. Proyecto, producto y operación
3. Ciclo de vida
4. Triple restricción
5. Stakeholders y roles
6. Gestión predictiva
7. Iteración e incremento
8. Origen de la agilidad
9. Manifiesto Ágil
10. Doce principios
11. Entrega de valor
12. Gestión del cambio
13. Elección del enfoque

*"Individuos e interacciones sobre procesos y herramientas. Software funcionando sobre documentación extensiva. Colaboración con el cliente sobre negociación contractual. Respuesta ante el cambio sobre seguimiento de un plan." — Manifiesto por el Desarrollo Ágil de Software, 2001*

## 1. Concepto de proyecto y gestión de proyectos

Antes de hablar de metodologías, marcos de trabajo o herramientas, es necesario detenerse en una pregunta que parece obvia pero que en la práctica profesional genera confusiones permanentes: ¿qué es exactamente un proyecto? El Project Management Institute (PMI), en su Guía del PMBOK, define un proyecto como "un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único". Esta definición, aunque breve, condensa dos atributos que son la clave de todo lo que vendrá después: la temporalidad y la unicidad. Un proyecto tiene un principio y un fin definidos —no se extiende indefinidamente en el tiempo— y produce algo que no existía antes de esa manera, con esas características particulares, para ese contexto específico.

Pensemos en ejemplos del mundo del software, que es el terreno donde se va a mover la mayor parte de esta materia. El desarrollo de una aplicación móvil de e-commerce para una pyme que hasta ahora solo vendía en un local físico es un proyecto: tiene un inicio (la firma del contrato o el kick-off), un conjunto de entregables únicos (la app, la integración con la pasarela de pagos, el panel de administración) y un final (la puesta en producción y el cierre formal). Lo mismo ocurre con la migración de un sistema legado de facturación a una arquitectura de microservicios, o con la implementación de un nuevo CRM en una empresa de seguros. En todos los casos hay un objetivo acotado, un conjunto de recursos asignados temporalmente y una fecha —aunque sea estimada— en la que el esfuerzo llega a su fin.

La gestión de proyectos, entonces, es la disciplina que se ocupa de aplicar conocimientos, habilidades, herramientas y técnicas a las actividades del proyecto para satisfacer sus requisitos. No es simplemente "organizar tareas": implica planificar el alcance, estimar tiempos y costos, gestionar riesgos, coordinar personas con intereses distintos, comunicar avances, tomar decisiones bajo incertidumbre y, en última instancia, entregar un resultado que genere valor para quien lo encargó. Un gestor de proyectos de software no es solamente quien arma un cronograma en una herramienta como Jira o MS Project: es quien articula la visión del negocio con la capacidad técnica del equipo, quien negocia expectativas con el

cliente y quien detecta a tiempo que un riesgo técnico puede convertirse en un problema de alcance o de presupuesto.

Es importante notar que la gestión de proyectos no es una invención reciente ni exclusiva de la industria del software. Sus raíces se remontan a la ingeniería civil y militar del siglo XX —la construcción de la represa Hoover en los años 30, los grandes programas aeroespaciales de la NASA en los 60— donde ya se usaban técnicas de planificación como los diagramas de Gantt (desarrollados por Henry Gantt a principios del siglo XX) o el método del camino crítico (Critical Path Method, CPM), creado en 1957 por DuPont y Remington Rand. La industria del software adoptó y adaptó estas prácticas a partir de los años 70 y 80, cuando los proyectos de desarrollo empezaron a crecer en tamaño y complejidad, y se hizo evidente que "programar sin más" no alcanzaba para entregar sistemas confiables, a tiempo y dentro de presupuesto.

#### CRITERIO PROFESIONAL:

Antes de elegir herramientas o ceremonias, formulá el resultado único que justifica el proyecto, el límite temporal del esfuerzo y el valor esperado. Si esos tres elementos no pueden explicarse con claridad, todavía no existe una base suficiente para planificar.

## 2. Proyecto, producto y operación: diferencias

Uno de los errores más comunes entre quienes se inician en la gestión de proyectos es confundir tres conceptos que están relacionados pero que no son lo mismo: proyecto, producto y operación. Distinguirlos con claridad es fundamental porque cada uno implica una lógica de gestión distinta, con objetivos, métricas de éxito y horizontes temporales diferentes.

Un **proyecto**, como vimos, es un esfuerzo temporal con un principio y un fin, orientado a crear algo único. Un **producto**, en cambio, es el resultado tangible o intangible que ese proyecto —o una sucesión de proyectos e iteraciones— genera, y que tiene un ciclo de vida propio que puede extenderse durante años: nace, crece, madura y eventualmente declina o se discontinúa. Pensemos en Spotify: el desarrollo inicial de la plataforma fue (o incluyó) uno o varios proyectos, pero Spotify como producto sigue vivo, evolucionando constantemente a través de nuevas funcionalidades, cada una de las cuales puede gestionarse como un proyecto o como parte de un flujo continuo de trabajo.

La **operación**, por su parte, es el conjunto de actividades permanentes y repetitivas que mantienen el negocio funcionando día a día. Si el proyecto fue desarrollar el sistema de facturación electrónica de una empresa, la operación es el proceso diario de emitir facturas, procesar pagos y responder consultas de soporte una vez que el sistema ya está en producción. La operación no tiene fecha de fin: es continua, cíclica, y su éxito se mide con

indicadores de eficiencia y estabilidad (tiempo de actividad del sistema, tickets resueltos por día, tasa de errores) más que con el cumplimiento de un alcance único.

En el mundo del desarrollo de software esta distinción se vuelve particularmente relevante cuando hablamos de equipos de producto versus equipos de proyecto. Un equipo de producto trabaja de manera continua sobre una misma base de código o plataforma, incorporando mejoras de forma incremental durante meses o años, sin que exista necesariamente una fecha de "cierre". Un equipo de proyecto, en cambio, se conforma para un objetivo acotado —por ejemplo, migrar la infraestructura a la nube— y se disuelve o se reasigna una vez que ese objetivo se cumple. Muchas organizaciones ágiles modernas tienden a organizar su trabajo alrededor de productos persistentes en lugar de proyectos discretos, precisamente porque el software rara vez "termina": se libera una versión, se recibe feedback, se ajusta, y el ciclo continúa indefinidamente mientras el producto siga vigente en el mercado.

Comparación entre proyecto, producto y operación

| Dimensión           | Proyecto                                                  | Producto                                                          | Operación                                                     |
|---------------------|-----------------------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------|
| Duración            | Temporal, con fecha de inicio y fin                       | Puede extenderse por años, mientras tenga vigencia en el mercado  | Continua, sin fecha de finalización prevista                  |
| Objetivo            | Entregar un resultado único y definido                    | Generar y sostener valor para los usuarios a lo largo del tiempo  | Mantener el funcionamiento estable del negocio día a día      |
| Equipo              | Se conforma y se disuelve según el proyecto               | Suele ser estable, dedicado de forma permanente al producto       | Personal fijo de soporte, mantenimiento y atención            |
| Ejemplo en software | Desarrollar una nueva app de pagos móviles                | La app de pagos evolucionando con nuevas features durante años    | Mesa de ayuda que atiende incidentes de la app en producción  |
| Métrica de éxito    | Cumplimiento de alcance, tiempo, costo y calidad pactados | Adopción, retención, satisfacción del usuario, ingresos generados | Disponibilidad, tiempo de respuesta, resolución de incidentes |



**Una misma solución, tres horizontes de gestión.** El proyecto crea un resultado acotado; el producto aprende y evoluciona; la operación sostiene disponibilidad y continuidad. La distinción cambia equipos, decisiones y métricas, aunque el sistema observado sea el mismo.

#### CRITERIO PROFESIONAL:

Definí la unidad de gestión antes de elegir sus métricas. Un hito de proyecto puede cumplirse y, aun así, el producto puede no ser adoptado o la operación puede resultar insostenible; cada horizonte necesita evidencia de éxito diferente.

## 3. Ciclo de vida de un proyecto

Todo proyecto, independientemente del enfoque metodológico que se adopte para gestionarlo, atraviesa un conjunto de fases que en conjunto conforman su ciclo de vida. Comprender estas fases es esencial porque permite anticipar qué tipo de actividades, decisiones y riesgos son propios de cada momento, y porque el ciclo de vida determina en buena medida cómo se organiza el trabajo, se asignan los recursos y se controla el avance.

### 3.1 Inicio

En esta fase se define la razón de ser del proyecto: qué problema resuelve, qué oportunidad de negocio aprovecha, quién lo patrocina y qué resultado de alto nivel se espera. Se elabora el acta de constitución del proyecto (project charter), se identifica a los primeros stakeholders y se realiza un análisis preliminar de viabilidad. En un proyecto de software, el inicio suele incluir una primera exploración técnica: ¿es viable construir esto con la tecnología

disponible?, ¿qué riesgos técnicos existen?, ¿cuál es el orden de magnitud del esfuerzo necesario?

## 3.2 Planificación

Aquí se detalla cómo se va a alcanzar el objetivo definido en el inicio. Se desarrolla el plan de gestión del proyecto, que incluye la definición del alcance, la estructura de descomposición del trabajo (EDT o WBS), el cronograma, el presupuesto, el plan de gestión de riesgos, el plan de comunicaciones y la definición de roles y responsabilidades. En proyectos de software con enfoque predictivo, esta fase puede ser extensa y detallada, documentando requisitos funcionales y no funcionales antes de escribir una sola línea de código. En enfoques ágiles, en cambio, la planificación es más liviana al inicio y se retoma de manera continua a lo largo de todo el proyecto, iteración tras iteración.

## 3.3 Ejecución

Es la fase en la que efectivamente se realiza el trabajo: se escribe el código, se diseñan las interfaces, se configuran los servidores, se prueban las funcionalidades. Es la fase que típicamente consume la mayor parte del presupuesto y del tiempo del proyecto, y en la que el equipo de desarrollo tiene el rol más protagónico. La calidad de la planificación previa impacta directamente en la fluidez de esta etapa: una mala definición de requisitos en la planificación se traduce, casi inevitablemente, en retrabajo durante la ejecución.

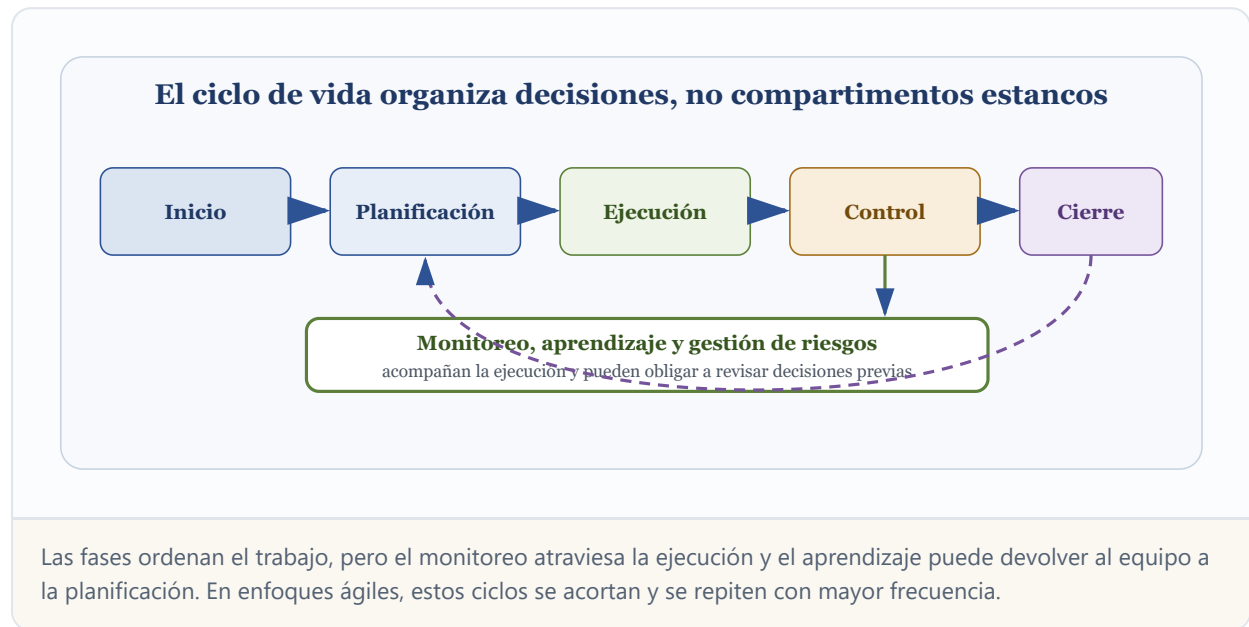
## 3.4 Monitoreo y control

Esta fase transcurre en paralelo con la ejecución (no es estrictamente secuencial) y consiste en comparar el avance real contra lo planificado, identificar desvíos, gestionar cambios de alcance, controlar riesgos emergentes y tomar acciones correctivas. En un proyecto de software esto incluye el seguimiento de métricas como velocidad del equipo, cantidad de bugs detectados, cobertura de pruebas o cumplimiento de hitos (milestones). En los enfoques ágiles, el monitoreo y control está integrado de manera mucho más estrecha en el propio ritmo de trabajo, a través de ceremonias como las reuniones diarias (daily stand-up) o las revisiones de sprint.

## 3.5 Cierre

Finalmente, el proyecto se cierra formalmente: se entrega el producto o resultado final, se documentan las lecciones aprendidas, se liberan los recursos asignados (personas, presupuesto, infraestructura) y se obtiene la aceptación formal del cliente o del sponsor. En software, el cierre puede incluir el pase a un equipo de mantenimiento u operaciones, la firma de un acta de aceptación y una instancia de retrospectiva final donde el equipo reflexiona sobre qué funcionó bien y qué se podría mejorar en proyectos futuros.

Es importante aclarar que estas cinco fases no son exclusivas de la gestión predictiva. Incluso en un proyecto gestionado con Scrum, que trabaja en ciclos cortos llamados sprints, existe un inicio del proyecto general, una planificación (que se repite en cada sprint planning), una ejecución (el sprint en sí), un monitoreo continuo (el daily y el seguimiento del backlog) y un cierre (tanto de cada sprint como del proyecto completo). Lo que cambia entre un enfoque y otro no es la existencia de estas fases, sino su duración, su grado de formalidad y la frecuencia con la que se repiten.



#### CRITERIO PROFESIONAL:

Usó las fases para reconocer qué decisión debe tomarse y qué evidencia la respalda. No las conviertas en barreras rígidas: si el monitoreo revela un supuesto inválido, volver a planificar es control responsable, no retroceso.

## 4. Alcance, tiempo, costo y calidad: la triple restricción

Uno de los conceptos más citados —y a veces mal utilizados— en la gestión de proyectos es el de la triple restricción (triple constraint), también conocido como el "triángulo de hierro" del proyecto. La idea central es sencilla: todo proyecto está limitado por tres variables que se encuentran en tensión permanente entre sí —alcance, tiempo y costo— y que la calidad del resultado depende del equilibrio que se logre entre ellas.

El **alcance** define qué es lo que el proyecto va a entregar: qué funcionalidades tendrá el sistema, qué características incluye el producto, qué está dentro y qué está fuera de los límites del trabajo acordado. El **tiempo** se refiere al cronograma: cuánto va a durar el proyecto, cuáles son sus hitos y su fecha de entrega. El **costo** representa el presupuesto disponible, que

en un proyecto de software suele estar dominado por el costo de las horas-persona del equipo, aunque también incluye licencias, infraestructura en la nube y otros insumos. La **calidad**, que algunos autores ubican como una cuarta restricción y otros como el resultado del equilibrio entre las tres primeras, se refiere al grado en que el producto entregado cumple con los requisitos funcionales y no funcionales esperados, y con los estándares profesionales de construcción de software (buenas prácticas de código, pruebas adecuadas, seguridad, performance).

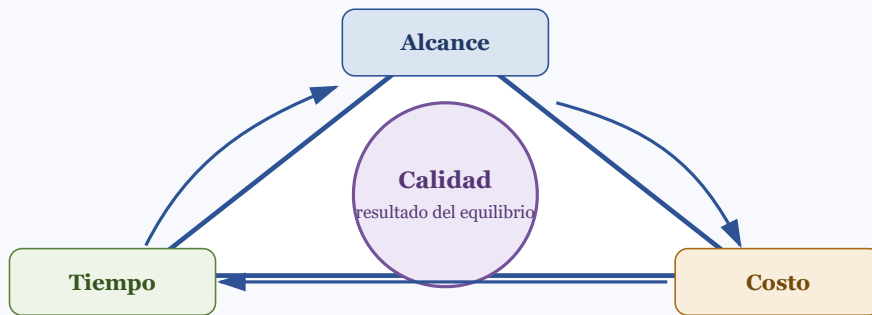
La lógica de la triple restricción establece que estas variables están interrelacionadas: si se modifica una, inevitablemente impacta en al menos una de las otras dos, y por lo tanto en la calidad del resultado final. Supongamos un proyecto de desarrollo de una app bancaria con un alcance, un cronograma y un presupuesto ya definidos. Si a mitad de camino el cliente solicita agregar una nueva funcionalidad de pagos por código QR (es decir, aumenta el alcance) sin mover la fecha de entrega ni el presupuesto, lo más probable es que la calidad se resienta: el equipo tendrá que apurar el desarrollo, recortar pruebas o acumular deuda técnica para cumplir con el nuevo alcance en el mismo tiempo. Alternativamente, el gestor de proyecto podría negociar una extensión del cronograma, un incremento del presupuesto (para sumar más desarrolladores, aunque esto no siempre acelera linealmente el trabajo, como advierte la conocida "ley de Brooks"), o la eliminación de otra funcionalidad de menor prioridad para compensar el nuevo pedido.

Esta tensión es precisamente uno de los puntos donde los enfoques predictivo y ágil difieren de manera más profunda. En el enfoque predictivo tradicional, lo habitual es fijar el alcance desde el inicio y dejar que el tiempo y el costo se ajusten en función de ese alcance (se estima cuánto va a llevar hacer "todo lo que se pidió"). En el enfoque ágil, la lógica se invierte: se fijan el tiempo (la duración del sprint, por ejemplo dos semanas) y el costo (el equipo, con su capacidad conocida), y es el alcance el que se ajusta de manera flexible, priorizando en cada iteración lo que aporta mayor valor dentro de esa capacidad fija. Esta inversión no es un detalle menor: es una de las claves para entender por qué los proyectos ágiles suelen adaptarse mejor a contextos de alta incertidumbre o de requisitos cambiantes.

La triple restricción: comportamiento en cada enfoque

| Variable                     | Enfoque predictivo (tradicional)                                | Enfoque ágil                                                      |
|------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|
| Alcance                      | Fijo, definido en detalle desde el inicio                       | Variable, se ajusta y prioriza en cada iteración                  |
| Tiempo                       | Estimado en función del alcance, puede extenderse               | Fijo, definido por la duración de cada iteración (sprint)         |
| Costo                        | Estimado en función del alcance y el cronograma                 | Relativamente fijo, definido por el tamaño y capacidad del equipo |
| Qué se adapta ante un cambio | Se renegocian tiempo y/o costo para sostener el alcance pactado | Se reprioriza o recorta alcance para sostener tiempo y costo      |

### Cambiar una restricción obliga a renegociar el sistema



Más alcance con igual tiempo y costo desplaza presión hacia calidad, riesgo o sostenibilidad.

La triple restricción no ofrece una fórmula automática: hace visible la negociación. Cuando una variable cambia, el equipo debe decidir cuál de las restantes se adapta y qué impacto acepta sobre calidad y riesgo.

**MITO:** Si el equipo trabaja más rápido, puede absorber cualquier aumento de alcance sin modificar el plan.

**REALIDAD:** La capacidad tiene límites y las horas extra no crean productividad lineal. Un cambio exige repriorizar alcance, renegociar tiempo o costo, o explicitar el riesgo que se está aceptando.

#### CRITERIO PROFESIONAL:

Frente a un pedido nuevo, no respondas primero con una fecha. Hacé visible qué variable cambia, qué alternativa se desplaza y qué efecto tendría cada opción sobre calidad, riesgo y valor.

## 5. Stakeholders y roles

Ningún proyecto existe en el vacío: siempre hay personas y organizaciones cuyos intereses se ven afectados, directa o indirectamente, por su desarrollo y su resultado. A este conjunto de individuos, grupos u organizaciones se los denomina **stakeholders** (interesados o partes interesadas), y su identificación y gestión adecuada es uno de los factores más determinantes del éxito o fracaso de un proyecto —de hecho, buena parte de los proyectos de software que fracasan lo hacen no por problemas técnicos, sino por una gestión deficiente de las expectativas y necesidades de sus stakeholders.

Entre los stakeholders más habituales de un proyecto de software encontramos al **sponsor** o patrocinador, que es quien autoriza y financia el proyecto, y ante quien el gestor de proyecto rinde cuentas sobre el avance y los resultados; el **cliente** o usuario final, que es quien va a utilizar el producto y cuya satisfacción es, en última instancia, la vara con la que se mide el

éxito; el **equipo de desarrollo**, compuesto por programadores, diseñadores UX/UI, testers, arquitectos de software y otros perfiles técnicos que construyen el producto; y actores externos como proveedores de infraestructura en la nube, entes reguladores (por ejemplo, en proyectos que manejan datos sensibles y deben cumplir con normativas de protección de datos) o competidores cuyo comportamiento en el mercado puede influir en las decisiones del proyecto.

Dentro de los marcos ágiles, y particularmente en Scrum, los roles se formalizan de una manera específica que vale la pena mencionar aquí porque será desarrollada en profundidad en módulos posteriores. El **Product Owner** es responsable de maximizar el valor del producto y de gestionar el backlog, actuando como la voz del cliente y del negocio dentro del equipo. El **Scrum Master** (o, en la terminología más reciente, "facilitador ágil") vela por que el equipo aplique correctamente las prácticas ágiles, remueve impedimentos y protege al equipo de interrupciones externas. El **equipo de desarrollo** es un grupo autoorganizado y multidisciplinario, responsable de convertir los elementos del backlog en incrementos de producto potencialmente entregables. Esta estructura de roles contrasta con la de un proyecto predictivo tradicional, donde suele haber un gerente de proyecto (project manager) con autoridad centralizada sobre la planificación, la asignación de tareas y el control del cronograma, y donde los roles técnicos están mucho más jerarquizados y especializados en funciones estancas (analista, diseñador, programador, tester, cada uno en una etapa distinta del proceso).

Gestionar adecuadamente a los stakeholders implica identificarlos temprano, comprender su nivel de interés y de poder o influencia sobre el proyecto, y definir estrategias de comunicación diferenciadas para cada grupo. Un stakeholder con alto poder y alto interés — como el sponsor que financia el proyecto— requiere una gestión cercana y participativa; un stakeholder con bajo poder pero alto interés —como un grupo de usuarios finales entusiasmados con el nuevo sistema— puede gestionarse manteniéndolo bien informado sin necesariamente involucrarlo en cada decisión. Esta clasificación, conocida habitualmente como matriz de poder-interés, es una herramienta simple pero muy efectiva para priorizar los esfuerzos de comunicación y gestión de expectativas a lo largo de todo el ciclo de vida del proyecto.

Otra herramienta ampliamente utilizada para clarificar responsabilidades entre stakeholders y miembros del equipo es la matriz RACI, cuyo nombre es un acrónimo de los cuatro tipos de participación que puede tener una persona respecto de una actividad o decisión del proyecto: Responsable (quien ejecuta el trabajo), Accountable o Aprobador (quien rinde cuentas finales por el resultado y tiene autoridad de decisión, idealmente una única persona por actividad), Consultado (quien debe ser consultado antes de tomar la decisión, típicamente porque aporta conocimiento experto) e Informado (quien debe ser notificado del resultado, aunque no participe de la decisión). En un proyecto de software, por ejemplo, la decisión de migrar la base de datos a un nuevo motor podría tener al arquitecto de software como Responsable de

ejecutar el análisis técnico, al gerente de tecnología como Aprobador final, al equipo de seguridad de la información como Consultado, y al equipo de soporte como Informado una vez tomada la decisión. Construir esta matriz al inicio del proyecto evita ambigüedades costosas sobre quién debe decidir qué, y es una práctica habitual tanto en proyectos predictivos como en proyectos ágiles de cierta escala, aunque en equipos ágiles pequeños y autoorganizados suele aplicarse de manera mucho más informal.



**El mapa de interesados es una arquitectura de decisiones.** Sponsor, usuarios, producto, facilitación, desarrollo, seguridad, proveedor y regulación no participan del mismo modo: algunos deciden, otros ejecutan, aportan conocimiento o necesitan información.

#### CRITERIO PROFESIONAL:

Gestionar stakeholders no significa comunicar más, sino diseñar la participación adecuada. Para cada decisión crítica, determinará quién aporta conocimiento, quién tiene autoridad, quién ejecuta y quién necesita conocer el resultado; una reunión masiva rara vez resuelve esa distinción.

## 6. Gestión tradicional de proyectos (enfoque predictivo o cascada)

El enfoque predictivo, también llamado en cascada (waterfall), es el modelo de gestión de proyectos más antiguo y, durante décadas, el dominante en la industria del software. Su lógica es lineal y secuencial: el proyecto avanza a través de fases claramente delimitadas —requisitos, diseño, implementación, verificación, mantenimiento— donde cada fase debe completarse y ser aprobada formalmente antes de que comience la siguiente. El nombre "cascada" hace referencia justamente a esa imagen: el trabajo cae de una etapa a

la siguiente como el agua que desciende por una serie de escalones, sin posibilidad —al menos en su formulación original— de volver atrás sin un costo considerable.

El origen formal de este modelo suele atribuirse a un artículo de Winston Royce publicado en 1970, titulado "Managing the Development of Large Software Systems". Es interesante —y a menudo señalado como una ironía histórica— que Royce en ese mismo artículo advertía sobre las limitaciones de un enfoque puramente secuencial y recomendaba iterar entre fases y validar tempranamente con prototipos; sin embargo, la industria adoptó masivamente la versión simplificada y estrictamente secuencial del modelo, probablemente porque encajaba muy bien con la lógica de contratos de precio fijo y con la necesidad de las organizaciones de contar con hitos de control claros y auditable.

En un proyecto gestionado bajo el enfoque cascada, el proceso típico comienza con una fase extensa de relevamiento y documentación de requisitos, en la que se intenta capturar de la manera más completa y precisa posible todo lo que el sistema deberá hacer. Sobre esa base se elabora un diseño detallado —arquitectura del sistema, modelo de datos, diseño de interfaces— que luego se traduce en código durante la fase de implementación. Una vez desarrollado el sistema completo (o una parte significativa de él), se ejecuta una fase de pruebas y verificación, y finalmente el sistema se libera a producción, entrando en una fase de mantenimiento que puede extenderse durante años.

Este enfoque presenta ventajas claras en determinados contextos: cuando los requisitos son estables y bien comprendidos desde el inicio, cuando existen restricciones contractuales o regulatorias que exigen documentación exhaustiva y aprobaciones formales en cada etapa (por ejemplo, en proyectos de software para la industria aeroespacial, médica o de defensa), o cuando el costo de introducir cambios en etapas avanzadas es extremadamente alto (como en sistemas embebidos en hardware que, una vez fabricado, es muy costoso modificar). La planificación detallada permite, además, estimar con mayor precisión presupuestos y plazos al inicio del proyecto, algo que muchas organizaciones —y especialmente los departamentos de compras o los organismos públicos que licitan desarrollos— valoran y en muchos casos exigen.

Sin embargo, el enfoque cascada muestra debilidades importantes cuando se aplica a proyectos de software con alta incertidumbre, donde los requisitos no pueden conocerse completamente por adelantado (porque dependen de aprendizajes que solo se obtienen viendo el producto funcionando) o donde el contexto de negocio cambia con rapidez. El problema central es que los errores de comprensión de los requisitos, si no se detectan hasta la fase de pruebas —es decir, casi al final del proyecto—, resultan extremadamente costosos de corregir, porque implican rehacer diseño e implementación ya completados. Numerosos estudios de la industria, como los históricos informes CHAOS del Standish Group, documentaron durante los años 90 y 2000 una tasa elevada de proyectos de software gestionados bajo cascada que se entregaban tarde, excedían el presupuesto o no satisfacían

las necesidades reales del negocio, lo cual fue uno de los motores que impulsó la búsqueda de enfoques alternativos.

## 6.1 El informe CHAOS y la evidencia empírica sobre el fracaso de proyectos

Resulta útil, para dimensionar el problema que dio origen a la búsqueda de alternativas al enfoque cascada, revisar algunos datos concretos. El Standish Group, una consultora estadounidense, comenzó a publicar en 1994 el llamado "CHAOS Report", un relevamiento periódico sobre el estado de los proyectos de tecnología de la información en Estados Unidos. Las primeras ediciones de este informe mostraron cifras alarmantes: en el relevamiento original de 1994, apenas alrededor de un 16% de los proyectos de software se consideraban exitosos (entregados a tiempo, dentro del presupuesto y con el alcance completo pactado), mientras que una proporción similar directamente fracasaba por completo (se cancelaba antes de terminar o el resultado nunca se usaba), y el resto se completaba mostrando desvíos significativos de alcance, tiempo o presupuesto, en una categoría que el informe denominó "challenged" (desafiados o comprometidos).

Si bien la metodología del informe CHAOS ha sido objeto de críticas académicas a lo largo de los años —por ejemplo, por definir el "éxito" de manera estrictamente ligada al cumplimiento de la triple restricción original, sin considerar si el producto generaba valor real—, estas cifras fueron ampliamente citadas durante los años 90 y 2000 como evidencia de que algo en la forma tradicional de gestionar proyectos de software no estaba funcionando. Ediciones posteriores del mismo informe, ya en la era de adopción masiva de metodologías ágiles, mostraron una mejora sostenida en las tasas de éxito de los proyectos que declaraban usar enfoques ágiles frente a los que seguían un enfoque predictivo puro, lo cual —sin ser una prueba causal definitiva, dado que intervienen muchas otras variables— reforzó la percepción de la industria de que la adaptabilidad ágil ofrecía ventajas tangibles en un dominio tan propenso a la incertidumbre como el desarrollo de software.

### CRITERIO PROFESIONAL:

Evaluá el enfoque predictivo por su ajuste al contexto, no por una caricatura. Es valioso cuando la estabilidad, la auditabilidad y el costo de cambio justifican decidir temprano; se vuelve riesgoso cuando esa certeza es solo una suposición.

## 7. Enfoques predictivos, iterativos e incrementales

Para comprender en profundidad el terreno sobre el que se asienta la agilidad, es necesario distinguir con precisión tres términos que suelen mencionarse juntos pero que

refieren a características distintas de cómo se organiza el trabajo de un proyecto: predictivo, iterativo e incremental. Estas categorías no son necesariamente excluyentes — de hecho, la mayoría de los marcos ágiles combinan características iterativas e incrementales— pero entenderlas por separado ayuda a evitar la simplificación de reducir todo a una dicotomía entre "cascada" y "ágil".

El enfoque **predictivo**, como vimos en el apartado anterior, se caracteriza por planificar el proyecto en detalle desde el comienzo, definiendo alcance, cronograma y presupuesto antes de iniciar la ejecución, y avanzando de manera lineal a través de fases secuenciales. El énfasis está puesto en la previsibilidad: se busca anticipar el mayor número posible de variables antes de comenzar a construir.

El enfoque **iterativo**, en cambio, organiza el trabajo en ciclos repetidos (iteraciones) durante los cuales se revisa y refina el producto o partes de él. En un desarrollo iterativo, el equipo puede construir una primera versión rudimentaria de una funcionalidad, obtener feedback, y luego volver sobre esa misma funcionalidad para mejorarla en una iteración posterior. La clave del enfoque iterativo es que permite aprender y ajustar el rumbo a medida que el proyecto avanza, en lugar de intentar definirlo todo de antemano. Un ejemplo clásico es el desarrollo de un algoritmo de recomendación: en la primera iteración se implementa una versión simple basada en popularidad, se mide su desempeño, y en iteraciones sucesivas se van incorporando variables más sofisticadas (historial del usuario, comportamiento de usuarios similares, contexto temporal) según lo que el aprendizaje de cada ciclo indique que aporta más valor.

El enfoque **incremental** organiza el trabajo en partes o "incrementos" que se van entregando de manera sucesiva, cada uno añadiendo funcionalidad nueva y utilizable sobre lo ya construido, hasta completar el producto final. A diferencia del enfoque iterativo, donde se vuelve sobre lo mismo para mejorarlo, en el enfoque incremental cada entrega agrega algo nuevo que antes no existía. Pensemos en el desarrollo de una plataforma de streaming de video: un primer incremento podría ser la posibilidad de ver contenido en un catálogo fijo; un segundo incremento agrega la función de listas personalizadas; un tercero suma recomendaciones; un cuarto incorpora la posibilidad de descargar contenido offline. Cada incremento es funcional por sí mismo y aporta valor al usuario, aun cuando el producto completo todavía no esté terminado.

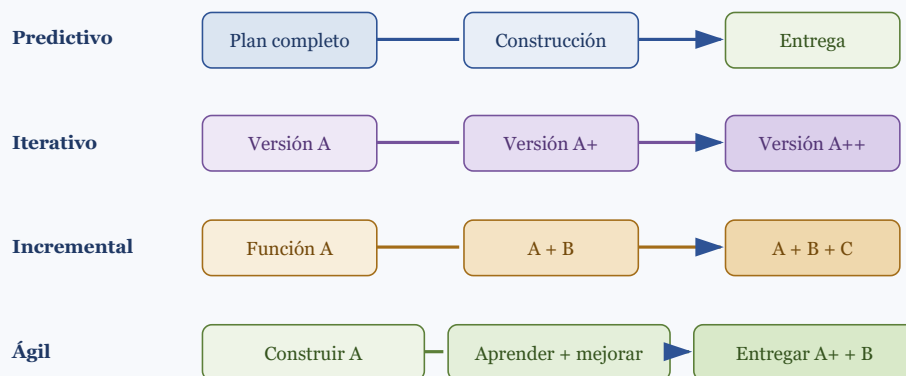
En la práctica, los marcos ágiles modernos —Scrum es el ejemplo paradigmático— combinan ambos enfoques: son iterativos porque trabajan en ciclos cortos y repetidos (los sprints) donde se revisa constantemente el rumbo del producto, y son incrementales porque cada sprint produce un incremento de producto potencialmente entregable, que se suma a lo construido en sprints anteriores. Esta combinación —iterativo-incremental— es lo que permite a los equipos ágiles aprender rápido (gracias a la naturaleza iterativa) y, al mismo

tiempo, mostrar avances tangibles y utilizables de manera constante (gracias a la naturaleza incremental).

#### Predictivo, iterativo e incremental: características centrales

| Enfoque     | Lógica de trabajo                                           | Cuándo aporta más valor                                                     | Ejemplo                                                       |
|-------------|-------------------------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------|
| Predictivo  | Planificación detallada al inicio, ejecución secuencial     | Requisitos estables, alta necesidad de previsibilidad contractual           | Desarrollo de firmware para un dispositivo médico certificado |
| Iterativo   | Ciclos repetidos que refinan el mismo producto o componente | Alta incertidumbre sobre la solución óptima, necesidad de aprender haciendo | Ajuste sucesivo de un algoritmo de recomendación              |
| Incremental | Entregas sucesivas que suman funcionalidad nueva y usable   | Necesidad de generar valor temprano y en partes                             | Lanzamiento progresivo de funciones en una app de streaming   |

#### Iterar mejora lo construido; incrementar amplía lo disponible



Una iteración revisa y mejora una solución; un incremento suma una capacidad utilizable. Un enfoque ágil combina ambos movimientos: aprende sobre lo construido y entrega valor adicional en ciclos breves.

#### CRITERIO PROFESIONAL:

No describas un proceso como ágil solo porque divide el cronograma en etapas. Preguntá si cada ciclo produce aprendizaje que cambia decisiones y si cada entrega agrega una capacidad realmente utilizable.

## 8. Origen de la agilidad

La agilidad no surgió de la nada ni fue el invento repentino de un grupo de personas en un fin de semana, aunque su momento fundacional formal —la redacción del Manifiesto

Ágil en 2001— sí ocurrió en apenas unos días. En realidad, la agilidad es la sistematización y el nombre que se le dio a un conjunto de prácticas e ideas que distintos equipos y consultores venían desarrollando y experimentando de manera más o menos independiente desde fines de los años 80 y durante los años 90, como respuesta a las limitaciones cada vez más evidentes del enfoque cascada aplicado al desarrollo de software.

Durante los años 90, varios profesionales del desarrollo de software, insatisfechos con los resultados que obtenían los proyectos gestionados bajo metodologías pesadas y altamente documentadas, comenzaron a proponer alternativas. Kent Beck desarrolló Extreme Programming (XP) en el contexto del proyecto de nómina de Chrysler (el "Chrysler Comprehensive Compensation System"), enfatizando prácticas técnicas como la programación en pares, el desarrollo guiado por pruebas (TDD) y las entregas frecuentes. Ken Schwaber y Jeff Sutherland formalizaron Scrum a mediados de la década, inspirados en un artículo de 1986 de Hirotaka Takeuchi e Ikujiro Nonaka publicado en Harvard Business Review, titulado "The New New Product Development Game", que describía cómo equipos de empresas como Honda, Canon o Fuji-Xerox trabajaban de manera multidisciplinaria y superpuesta —en lugar de secuencial— tomando como metáfora el rugby, donde todo el equipo avanza junto ("scrum") en lugar de pasarse el balón de un especialista a otro. Otros marcos como Crystal (Alistair Cockburn), Feature-Driven Development (FDD) y el desarrollo adaptativo de software (Jim Highsmith) surgieron en paralelo, cada uno con su propio énfasis pero compartiendo una preocupación común.

Esa preocupación común era, en esencia, la constatación de que el software es un producto de naturaleza distinta a otros productos de ingeniería. A diferencia de un puente o un edificio, cuyo diseño puede especificarse con un alto grado de certeza antes de construirlo (las leyes de la física no cambian), el software se construye para resolver problemas de negocio que frecuentemente son ambiguos, cambiantes y solo se comprenden en profundidad a medida que se interactúa con una primera versión del producto. Además, el contexto de fines de los 90 y comienzos de los 2000 estaba marcado por la explosión de internet, que aceleró drásticamente los ciclos de negocio: las empresas ya no podían darse el lujo de esperar dieciocho meses o dos años para lanzar un producto, porque el mercado y la competencia se movían mucho más rápido que eso.

En febrero de 2001, diecisiete de estos profesionales —entre ellos Kent Beck, Ward Cunningham, Martin Fowler, Jim Highsmith, Jon Kern, Brian Marick, Robert C. Martin, Ken Schwaber, Jeff Sutherland y Alistair Cockburn, entre otros— se reunieron en un resort de esquí en Snowbird, Utah, con el objetivo de encontrar puntos en común entre sus distintos enfoques y, eventualmente, ponerles un nombre compartido. De ese encuentro surgió el término "ágil" (que se prefirió a otras alternativas discutidas, como "adaptativo" o "liviano") y el documento que hoy se conoce como el Manifiesto por el Desarrollo Ágil de Software, que analizaremos en detalle en el próximo apartado.



**Snowbird fue un punto de convergencia, no un comienzo desde cero.** La escena representa cómo prácticas y experiencias previas se pusieron en común hasta formular un conjunto compartido de valores.

#### CRITERIO PROFESIONAL:

Cuando evalúes una práctica ágil, busca el problema que intentaba resolver y la evidencia que genera. Copiar una ceremonia sin comprender su propósito histórico suele conservar la forma y perder la capacidad de adaptación.

## 9. El Manifiesto Ágil

El Manifiesto Ágil es, sin dudas, el documento fundacional de todo el movimiento ágil, y su redacción —breve, directa, casi provocadora para la época— explica en buena medida por qué logró una difusión tan amplia en la industria del software durante las dos décadas siguientes. El manifiesto no propone una metodología específica ni un conjunto de herramientas: propone un cambio de mentalidad, expresado a través de cuatro valores centrales y doce principios que los desarrollan.

El texto completo de los cuatro valores, tal como fue redactado en 2001, dice así: "Estamos descubriendo formas mejores de desarrollar software tanto por nuestra propia experiencia como ayudando a terceros. A través de este trabajo hemos llegado a valorar: Individuos e interacciones sobre procesos y herramientas. Software funcionando sobre documentación extensiva. Colaboración con el cliente sobre negociación contractual. Respuesta ante el cambio sobre seguimiento de un plan. Es decir, aunque valoramos los elementos de la derecha, valoramos más los de la izquierda."

Esta última frase —"aunque valoramos los elementos de la derecha, valoramos más los de la izquierda"— es clave y frecuentemente se pasa por alto o se malinterpreta. El manifiesto no dice que los procesos, las herramientas, la documentación, los contratos o los planes no tengan valor: dice que, ante una situación de tensión entre ambos elementos de cada par, el juicio ágil prioriza el de la izquierda. No es un manifiesto "anti-documentación" ni "anti-planificación": es un manifiesto que establece un orden de prioridades cuando esos elementos entran en conflicto.

#### Los cuatro valores del Manifiesto Ágil

| Valor priorizado            | Valor secundario (no descartado) | Qué significa en la práctica                                                                                                                                                                          |
|-----------------------------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Individuos e interacciones  | Procesos y herramientas          | Las personas y su comunicación directa resuelven problemas mejor que la sola aplicación mecánica de un proceso o una herramienta, aunque procesos y herramientas sigan siendo útiles como soporte.    |
| Software funcionando        | Documentación extensiva          | Un incremento de producto que funciona y puede mostrarse al cliente vale más como evidencia de avance real que un documento extenso que describe lo que se planea construir.                          |
| Colaboración con el cliente | Negociación contractual          | Trabajar codo a codo con el cliente durante todo el proyecto, ajustando el rumbo según su feedback, genera mejores resultados que limitarse a cumplir literalmente lo firmado en un contrato inicial. |
| Respuesta ante el cambio    | Seguimiento de un plan           | La capacidad de adaptarse cuando cambian las condiciones o el conocimiento del problema es más valiosa que la rigidez de cumplir un plan que quedó desactualizado.                                    |

Vale la pena detenerse en cada uno de estos cuatro valores con algo más de profundidad, porque cada uno responde a un problema concreto que los firmantes del manifiesto habían identificado en su experiencia profesional. El primer valor, individuos e interacciones sobre procesos y herramientas, surge de la observación de que muchos proyectos fracasaban no por falta de metodología formal, sino por falta de comunicación fluida entre las personas involucradas: equipos que se comunicaban únicamente a través de documentos formales, con largos tiempos de espera entre una consulta y su respuesta, perdían la posibilidad de resolver ambigüedades con rapidez. Un equipo pequeño que se sienta junto al Product Owner y conversa directamente sobre un requisito ambiguo resuelve esa ambigüedad en minutos; el mismo problema, canalizado a través de un proceso formal de solicitud de cambios documentado, puede tardar semanas.

El segundo valor, software funcionando sobre documentación extensiva, responde a la constatación de que la documentación, aunque útil, no es el objetivo del proyecto: el objetivo es un producto que funcione y resuelva el problema del cliente. Documentos extensos de requisitos y diseño pueden dar una falsa sensación de control y avance, cuando en realidad no

garantizan que el software resultante funcione correctamente ni que responda a lo que el cliente realmente necesita. Mostrar software funcionando, aunque sea parcial, es una forma mucho más honesta y verificable de medir el progreso real de un proyecto.

El tercer valor, colaboración con el cliente sobre negociación contractual, apunta a superar la lógica adversarial que muchas veces se establece entre el proveedor de desarrollo y el cliente cuando el vínculo se rige exclusivamente por un contrato cerrado de alcance fijo. En esa lógica, cualquier cambio solicitado por el cliente se convierte en una negociación de "orden de cambio" con implicancias de costo y tiempo, lo cual desalienta la adaptación y genera fricción. La propuesta ágil es construir una relación de colaboración continua, donde el cliente participa activamente durante todo el desarrollo —revisando incrementos, dando feedback, reordenando prioridades— en lugar de limitarse a firmar un contrato al inicio y recibir el producto terminado al final.

El cuarto valor, respuesta ante el cambio sobre seguimiento de un plan, sintetiza quizás el espíritu más profundo de la agilidad: reconocer que, en un entorno de alta incertidumbre como el desarrollo de software, ningún plan elaborado al inicio del proyecto puede anticipar con precisión todo lo que va a ocurrir. Aferrarse rígidamente a ese plan inicial, incluso cuando la realidad ha demostrado que estaba equivocado o desactualizado, es una fuente frecuente de fracasos de proyecto. La agilidad propone, en cambio, tratar al cambio no como una amenaza a evitar sino como una fuente de ventaja competitiva: la capacidad de adaptarse rápido a nueva información es, en sí misma, un activo estratégico.

**MITO:** El Manifiesto Ágil propone trabajar sin procesos, documentación, contratos ni planes.

**REALIDAD:** Reconoce el valor de todos esos elementos, pero establece una prioridad cuando entran en tensión. La práctica profesional consiste en producir la evidencia y la documentación necesarias sin sustituir con ellas la colaboración, el aprendizaje ni el producto funcionando.

**CRITERIO PROFESIONAL:**

Aplicá los valores como criterios de desempate. Si una práctica formal mejora la coordinación o reduce un riesgo real, mantenela; si solo produce cumplimiento aparente y retrasa feedback útil, rediseñala.

## 10. Los 12 principios ágiles

Además de los cuatro valores, el Manifiesto Ágil incluye doce principios que los desarrollan y les dan un carácter más operativo, más cercano a la práctica cotidiana de un equipo de desarrollo. Estos principios no deben leerse como una lista de reglas aisladas,

sino como un conjunto coherente que, en su totalidad, describe una forma particular de entender y gestionar el trabajo de desarrollo de software.

- 1. Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor.** Este principio invierte la lógica tradicional de "entregar todo al final": propone que el cliente empiece a recibir y a usar partes del producto lo antes posible, de manera que el valor comience a generarse tempranamente y no recién al cierre del proyecto, que en un enfoque tradicional puede estar meses o años después del inicio.
- 2. Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.** A diferencia del enfoque cascada, donde un cambio tardío se percibe como una amenaza al cronograma y al presupuesto, la agilidad concibe al cambio como una oportunidad: si el mercado cambió o el cliente aprendió algo nuevo, adaptarse a esa información es más valioso que sostener ciegamente un plan desactualizado.
- 3. Entregamos software funcionando frecuentemente, en periodos de un par de semanas hasta un par de meses, con preferencia por los periodos breves.** Este principio establece el ritmo cadenciado que caracteriza a los marcos ágiles como Scrum, cuyos sprints suelen durar entre una y cuatro semanas. Entregar con esta frecuencia permite obtener feedback real del cliente de manera constante, en lugar de esperar a una única entrega final.
- 4. Las personas del negocio y los desarrolladores deben trabajar juntas de forma cotidiana a través del proyecto.** Se enfatiza aquí la colaboración diaria y directa entre quienes conocen el negocio (por ejemplo, el Product Owner) y quienes construyen la solución técnica, evitando la separación estricta entre "quien pide" y "quien ejecuta" que caracterizaba a muchos proyectos tradicionales, donde el contacto era esporádico y mediado por documentos.
- 5. Los proyectos se construyen en torno a individuos motivados. Hay que darles el entorno y el apoyo que necesitan, y confiar en ellos para que hagan el trabajo.** Este principio pone el foco en la motivación intrínseca de las personas y en la confianza como base de la gestión, en contraposición a modelos de control estricto y microgestión que asumen que los equipos necesitan supervisión constante para rendir adecuadamente.
- 6. El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.** Aunque este principio fue escrito antes de la masificación del trabajo remoto y de las herramientas de videoconferencia, su espíritu se mantiene vigente: la comunicación sincrónica y directa —

hoy incluyendo videollamadas— resuelve ambigüedades con más rapidez y riqueza que la comunicación exclusivamente escrita y asincrónica.

7. **El software funcionando es la principal medida de progreso.** Este principio retoma el segundo valor del manifiesto y lo convierte en criterio de medición: no se mide el avance de un proyecto ágil por la cantidad de documentos producidos o por el porcentaje de tareas "completadas" en un cronograma, sino por la cantidad de funcionalidad real, probada y utilizable que el equipo ha entregado.
8. **Los procesos ágiles promueven el desarrollo sostenible. Los promotores, desarrolladores y usuarios deberían ser capaces de mantener un ritmo constante de forma indefinida.** Se rechaza aquí la lógica de "sprints finales" con sobrecarga de trabajo (crunch) que históricamente ha caracterizado a muchos proyectos de software bajo presión de entrega. La sostenibilidad del ritmo de trabajo es, según este principio, condición necesaria para la calidad y la productividad a largo plazo.
9. **La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.** La velocidad de entrega no debe lograrse a costa de la calidad técnica: por el contrario, un código bien diseñado, con buenas prácticas de arquitectura y pruebas automatizadas, es lo que permite sostener un ritmo ágil de entregas en el tiempo, evitando que la acumulación de deuda técnica frene al equipo en el futuro.
10. **La simplicidad, o el arte de maximizar la cantidad de trabajo no realizado, es esencial.** Este principio, quizás el más contraintuitivo a primera vista, propone que buena parte del valor de un equipo ágil está en decidir qué NO hacer: evitar construir funcionalidades que nadie va a usar, evitar sobre-diseñar soluciones para problemas que todavía no existen, y concentrar el esfuerzo en lo que realmente aporta valor al cliente en el momento presente.
11. **Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados.** Se confía en la capacidad del equipo técnico —que tiene el conocimiento más cercano al detalle de la implementación— para tomar las mejores decisiones de diseño, en lugar de imponer esas decisiones desde una autoridad externa jerárquica y alejada del trabajo cotidiano de construcción del software.
12. **A intervalos regulares, el equipo reflexiona sobre cómo ser más efectivo y ajusta su comportamiento en consecuencia.** Este principio da fundamento a la práctica de la retrospectiva, una de las ceremonias centrales de Scrum y de otros marcos ágiles, en la que el equipo se detiene periódicamente a evaluar su propio proceso de trabajo y a proponer mejoras concretas para el ciclo siguiente, instalando una cultura de mejora continua.

#### CRITERIO PROFESIONAL:

No audites la agilidad contando ceremonias. Observá si el equipo entrega valor con frecuencia, incorpora feedback, sostiene calidad técnica y ajusta su forma de trabajo con evidencia; los principios se reconocen en esos comportamientos.

## 11. Entrega de valor como eje central

Si hay una idea que atraviesa transversalmente tanto los cuatro valores como los doce principios del Manifiesto Ágil, esa idea es la entrega de valor como criterio último de éxito de un proyecto. En la gestión ágil, el valor no se define en abstracto: se define siempre en relación con el cliente o usuario final, y se entiende como la utilidad real que una funcionalidad, un producto o un incremento le aporta a quien lo va a usar o a quien financia el proyecto.

Este eje en el valor implica un cambio profundo respecto de la manera tradicional de medir el éxito de un proyecto. En un enfoque predictivo clásico, el éxito suele medirse en función del cumplimiento del "triángulo de hierro": ¿se entregó dentro del alcance pactado, en el tiempo estimado y dentro del presupuesto asignado? Esta métrica, aunque relevante, dice muy poco sobre si lo que se construyó efectivamente sirve, se usa y genera el impacto de negocio esperado. Existen numerosos casos documentados de proyectos que se completaron "a tiempo y en presupuesto" según los criterios tradicionales, pero cuyo producto resultante fue un fracaso comercial porque no resolvía el problema real de los usuarios o porque el mercado había cambiado durante el desarrollo.

La agilidad propone, en cambio, medir el éxito en función del valor entregado de manera continua: ¿cuánto valor generó el equipo en este sprint?, ¿qué funcionalidad entregada está siendo efectivamente usada por los clientes?, ¿qué feedback obtuvimos y qué aprendimos sobre lo que el mercado realmente necesita? Esta orientación explica por qué en los marcos ágiles el backlog de producto se prioriza constantemente en función del valor esperado de cada elemento, y por qué el rol del Product Owner —cuya responsabilidad central es justamente maximizar ese valor— es tan determinante en la efectividad de un equipo ágil.

Priorizar por valor implica también aceptar que no todo lo que se podría construir merece ser construido, al menos no en el corto plazo. Técnicas como el análisis costo-beneficio, la priorización MoSCoW (must have, should have, could have, won't have) o el uso de métricas de impacto de negocio (por ejemplo, cuánto podría incrementar una funcionalidad la tasa de conversión de un e-commerce) son herramientas habituales para tomar estas decisiones de priorización de manera más objetiva y menos guiada por intuiciones o preferencias personales de quienes participan del proyecto.

**CRITERIO PROFESIONAL:**

Tratá el valor como una hipótesis que debe medirse. Antes de priorizar una funcionalidad, explicitá para quién crea valor, qué comportamiento esperás modificar y qué señal permitiría decidir si la inversión funcionó.

## 12. Gestión del cambio en contextos ágiles vs. tradicionales

La manera en que cada enfoque de gestión de proyectos concibe y gestiona el cambio es, quizás, el punto de mayor contraste entre la gestión tradicional y la gestión ágil, y merece un desarrollo específico más allá de lo ya mencionado en apartados anteriores. En la gestión tradicional, el cambio se gestiona a través de un proceso formal llamado control de cambios (change control), que típicamente involucra la presentación de una solicitud de cambio, su análisis de impacto sobre alcance, tiempo y costo, su aprobación por parte de un comité de control de cambios (Change Control Board) y, recién entonces, su incorporación al plan del proyecto. Este proceso tiene la virtud de mantener la disciplina y la trazabilidad del proyecto, evitando lo que se conoce como "scope creep" o expansión descontrolada del alcance, pero tiene como contrapartida una lentitud considerable: entre la identificación de la necesidad de un cambio y su efectiva incorporación al proyecto pueden pasar semanas.

En la gestión ágil, el cambio no requiere de un proceso formal de aprobación separado: se gestiona de manera natural a través de la reprogramación continua del backlog de producto. Si surge una nueva necesidad o cambia una prioridad, el Product Owner simplemente reordena el backlog, y ese cambio se incorpora al trabajo del equipo en la siguiente oportunidad de planificación (típicamente, el siguiente sprint). No existe la necesidad de un comité de aprobación ni de un análisis formal de impacto sobre un plan extenso, porque el plan ágil, por diseño, es de corto alcance y se revisa constantemente.

Esta diferencia tiene implicancias profundas sobre el tipo de proyectos para los que cada enfoque resulta más adecuado. En contextos donde la incertidumbre es alta —por ejemplo, el desarrollo de un producto digital nuevo, dirigido a un mercado que todavía no se comprende completamente, y donde el equipo espera aprender y ajustar el rumbo a medida que avanza— la capacidad de incorporar cambios de manera ágil y sin fricción es una ventaja decisiva. En contextos donde la incertidumbre es baja y los requisitos están bien establecidos y son estables —por ejemplo, el desarrollo de un sistema que debe cumplir con una especificación regulatoria fija y auditable— el control de cambios formal de la gestión tradicional puede ser, en cambio, una salvaguarda necesaria antes que un obstáculo.

Es importante también señalar que "gestionar el cambio de manera ágil" no equivale a "no tener ningún control sobre el alcance del proyecto". Los marcos ágiles bien implementados mantienen disciplina: el Product Owner es responsable de que el backlog no crezca de manera desordenada, las estimaciones del equipo (por ejemplo, mediante story points) permiten calcular la capacidad real disponible en cada sprint, y las revisiones de sprint (sprint reviews) exponen el avance real a los stakeholders de manera transparente y periódica. La flexibilidad ágil ante el cambio no es sinónimo de ausencia de gestión: es una forma distinta, más frecuente y más liviana, de ejercer esa gestión.

**CRITERIO PROFESIONAL:**

Aceptar el cambio no significa incorporarlo de inmediato. Registrá la necesidad, evaluá su valor y su urgencia, comparala con el trabajo ya priorizado y decidí en el momento de planificación adecuado sin desestabilizar el objetivo vigente.

## **13. Enfoques ágiles, tradicionales e híbridos: comparación y cuándo usar cada uno**

Después de haber recorrido los fundamentos del enfoque predictivo y de la agilidad, es momento de sintetizar la comparación entre ambos —y de incorporar una tercera categoría cada vez más presente en la práctica profesional: los enfoques híbridos, que combinan elementos de gestión predictiva y ágil según las necesidades específicas de cada proyecto u organización.

Ningún enfoque es intrínsecamente superior al otro en abstracto: la elección adecuada depende del contexto del proyecto, y en particular de dos variables clave que conviene evaluar de manera explícita antes de decidir cómo gestionar un proyecto. La primera es el grado de incertidumbre o claridad de los requisitos: ¿sabemos con precisión qué hay que construir, o vamos a descubrirlo a medida que avanzamos? La segunda es el costo del cambio en etapas avanzadas: ¿qué tan costoso —en tiempo, dinero o riesgo— es corregir un error o incorporar un cambio una vez que el desarrollo está avanzado?

Cuando los requisitos son claros y estables, y el costo de cambiar en etapas avanzadas es alto, el enfoque predictivo suele ser la opción más razonable: permite planificar con precisión, cumplir con exigencias contractuales o regulatorias, y minimizar sorpresas. Cuando los requisitos son inciertos o van a evolucionar durante el proyecto, y el costo de aprender y ajustar el rumbo es relativamente bajo (como suele ocurrir en el desarrollo de software, donde iterar sobre el código es, en general, mucho menos costoso que iterar sobre una construcción física), el enfoque ágil se vuelve claramente preferible, porque permite validar hipótesis tempranamente y evitar invertir grandes esfuerzos en construir algo que termine no siendo lo que el cliente necesitaba.

Los **enfoques híbridos** han ganado terreno particularmente en organizaciones grandes y en proyectos complejos que combinan componentes de distinta naturaleza. Un ejemplo típico es un proyecto de implementación de un sistema ERP para una empresa industrial: el componente de infraestructura y configuración base del sistema puede gestionarse con un enfoque más predictivo (porque involucra contratos con proveedores, licencias y una arquitectura relativamente estable), mientras que el desarrollo de integraciones y personalizaciones específicas puede gestionarse de manera ágil, con sprints iterativos que permiten ajustar el desarrollo en función del feedback de los usuarios del negocio. Otro ejemplo frecuente es el uso de marcos como el PMBOK (que desde su séptima edición incorpora explícitamente principios y prácticas ágiles junto con los tradicionales) o marcos de escalado ágil como SAgile (Scaled Agile Framework), que combinan una capa de planificación de más largo plazo y mayor formalidad —a nivel de programa o de portafolio— con ejecución ágil a nivel de equipo.

#### Comparación integral: enfoque predictivo, ágil e híbrido

| Criterio             | Predictivo                                                    | Ágil                                                       | Híbrido                                                              |
|----------------------|---------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------|
| Requisitos           | Definidos y documentados en detalle al inicio                 | Emergen y se refinan durante el proyecto                   | Parte definida al inicio, parte emergente                            |
| Planificación        | Extensa, detallada, de largo plazo                            | Liviana al inicio, continua a lo largo del proyecto        | Planificación macro de largo plazo + planificación ágil de detalle   |
| Gestión del cambio   | Formal, a través de control de cambios                        | Continua, integrada en la repriorización del backlog       | Control de cambios para hitos macro, flexibilidad en la ejecución    |
| Entregas al cliente  | Generalmente una única entrega al final del proyecto          | Frecuentes, cada una o dos semanas (sprints)               | Hitos formales combinados con entregas incrementales                 |
| Contextos favorables | Requisitos estables, alta exigencia regulatoria o contractual | Alta incertidumbre, necesidad de aprender y adaptar rápido | Proyectos grandes y complejos con componentes de distinta naturaleza |
| Ejemplo de proyecto  | Software embebido para un dispositivo médico certificado      | Desarrollo de una nueva app de consumo masivo              | Implementación de un ERP con integraciones a medida                  |

En definitiva, la decisión sobre qué enfoque adoptar —predictivo, ágil o híbrido— no debería tomarse por moda ni por convicción ideológica, sino como resultado de un análisis honesto del contexto del proyecto: el nivel de certeza sobre lo que hay que construir, la volatilidad del entorno de negocio, las exigencias contractuales o regulatorias, la cultura de la organización y la capacidad y madurez del equipo para trabajar de manera autoorganizada. Un buen gestor de proyectos de software del siglo XXI no es aquel que domina un único enfoque, sino aquel capaz de diagnosticar correctamente qué combinación de prácticas predictivas y ágiles resulta

más adecuada para cada proyecto particular, y de adaptar esa combinación a medida que el propio proyecto evoluciona.

Un marco sencillo y muy utilizado en la industria para apoyar este diagnóstico es el llamado modelo Cynefin, desarrollado por Dave Snowden, que clasifica los problemas y contextos de decisión en cuatro dominios: simple (o claro), donde la relación causa-efecto es evidente y existen mejores prácticas conocidas; complicado, donde la relación causa-efecto existe pero requiere de análisis experto para descubrirla; complejo, donde la relación causa-efecto solo puede comprenderse en retrospectiva, después de actuar y observar el resultado; y caótico, donde no hay una relación causa-efecto identificable en el momento y se requiere actuar primero para luego poder analizar. Aplicado a la gestión de proyectos, un problema "complicado" —como configurar la infraestructura de red de un centro de datos según especificaciones técnicas conocidas— se presta bien a un enfoque predictivo apoyado en expertos. Un problema "complejo" —como diseñar la experiencia de usuario óptima para una audiencia cuyo comportamiento todavía no se conoce con certeza— se beneficia mucho más de un enfoque ágil de prueba, observación y ajuste, porque en un dominio complejo no existe forma de "planificar la solución correcta" de antemano: solo se puede aprender experimentando en ciclos cortos y observando qué funciona.

Esta idea conecta con una reflexión final que conviene tener presente a lo largo de toda la materia: la agilidad no es un fin en sí mismo ni una religión metodológica, sino una respuesta racional a un tipo particular de problema de gestión, el que se presenta cuando la incertidumbre es alta y el aprendizaje solo puede lograrse construyendo y observando. Comprender esta lógica de fondo —y no simplemente memorizar los nombres de las ceremonias o los roles de Scrum— es lo que permitirá, en los módulos siguientes de esta materia, aplicar los marcos ágiles con criterio profesional genuino, en lugar de aplicarlos de manera mecánica o dogmática en contextos donde quizás no sean la mejor opción disponible.

### 13.1 Caso resuelto: incorporar pagos QR en una app bancaria

Retomemos la app bancaria presentada al estudiar la triple restricción. El proyecto debe habilitar una nueva versión en doce semanas por un compromiso regulatorio. El equipo estable tiene seis integrantes y no puede ampliarse durante ese período. A mitad del trabajo, las pruebas con clientes revelan que el pago mediante código QR tendría alto valor, pero esa funcionalidad no estaba incluida. Los siguientes datos son **supuestos didácticos** para mostrar cómo se razona la decisión.

- 1 **Separar restricciones negociables y no negociables.** La fecha regulatoria y el equipo disponible fijan tiempo y costo. También son obligatorios los requisitos de seguridad, trazabilidad y cumplimiento. El alcance comercial restante sí puede reordenarse.

- 2 Clasificar la incertidumbre.** Las reglas regulatorias están documentadas y requieren evidencia auditable, por lo que admiten planificación predictiva. La experiencia de pago QR contiene incertidumbre de uso: hay que observar si las personas comprenden el escaneo, la confirmación y los errores.
- 3 Elegir un enfoque híbrido deliberado.** Se mantiene un plan de hitos para certificación y cumplimiento, mientras la experiencia QR se desarrolla en iteraciones de dos semanas. Doce semanas permiten seis ciclos de construcción, prueba y ajuste.
- 4 Renegociar alcance con transparencia.** El Product Owner compara el valor esperado del pago QR con funcionalidades todavía no iniciadas. Se incorpora QR y se posterga un módulo de cupones de menor impacto; no se oculta el cambio dentro de la carga del equipo.
- 5 Definir evidencia de progreso.** Para cumplimiento, la evidencia son controles implementados, pruebas y trazabilidad aprobadas. Para QR, son incrementos utilizables revisados con clientes y métricas de tarea como finalización, errores y abandono.
- 6 Revisar la decisión.** Si las pruebas tempranas muestran que QR no resuelve el problema o introduce un riesgo que no puede mitigarse, el backlog se vuelve a priorizar. La decisión inicial no se protege por orgullo: se contrasta con evidencia.

#### Resultado de la decisión sobre el proyecto bancario

| Dimensión | Decisión                            | Fundamento                                                                          |
|-----------|-------------------------------------|-------------------------------------------------------------------------------------|
| Tiempo    | Mantener doce semanas               | Es una restricción regulatoria externa.                                             |
| Costo     | Mantener el equipo de seis personas | Sumar personas a mitad del proyecto no garantiza aceleración y agrega coordinación. |
| Alcance   | Incorporar QR y postergar cupones   | Se intercambia alcance según valor, sin degradar controles obligatorios.            |
| Enfoque   | Híbrido                             | Combina hitos auditables con aprendizaje iterativo sobre la experiencia.            |
| Calidad   | No negociable                       | Seguridad, pruebas y trazabilidad no se usan como variable de ajuste.               |

#### CRITERIO PROFESIONAL:

Elegí el enfoque por componentes de incertidumbre y riesgo, no solo por la etiqueta general del proyecto. Un mismo producto puede requerir control predictivo para obligaciones estables y ciclos ágiles para decisiones que dependen de aprendizaje.

## Recursos audiovisuales sugeridos

#### **The Agile Manifesto - 4 Agile Values**

**Explained** ↗, de CollabNet VersionOne. Se recomienda para volver sobre la relación “valoramos más” sin interpretar que los elementos de la derecha carecen de valor.

#### **Agile vs Waterfall: Waterfall Wins! + CHEAT SHEET** ↗, de Development That Pays.

Se recomienda porque muestra, mediante un caso concreto, que la previsibilidad del contexto puede volver razonable un enfoque predictivo y obliga a justificar la elección metodológica.

## Síntesis del módulo

A lo largo de este módulo recorrimos los cimientos conceptuales sobre los que se construye toda la materia de Gestión de Proyectos Ágiles. Partimos de una definición precisa de qué es un proyecto —un esfuerzo temporal orientado a un resultado único— y lo distinguimos con claridad de los conceptos de producto y de operación, dos categorías con las que suele confundirse pero que responden a lógicas de gestión distintas. Revisamos el ciclo de vida típico de cualquier proyecto (inicio, planificación, ejecución, monitoreo y control, cierre), y analizamos la triple restricción de alcance, tiempo y costo, junto con la calidad, mostrando cómo su equilibrio se gestiona de manera radicalmente distinta en un enfoque predictivo frente a uno ágil.

Estudiamos también el rol central de los stakeholders y los distintos roles que participan de un proyecto de software, tanto en su versión tradicional (gerente de proyecto, equipo técnico jerarquizado) como en su versión ágil (Product Owner, Scrum Master, equipo autoorganizado). Analizamos en profundidad el enfoque predictivo o cascada, sus orígenes históricos, sus fortalezas en contextos de requisitos estables y sus limitaciones frente a la incertidumbre, y distinguimos con precisión los enfoques predictivo, iterativo e incremental, mostrando cómo los marcos ágiles modernos combinan estos dos últimos.

Finalmente, reconstruimos el origen histórico de la agilidad como respuesta a las limitaciones del enfoque tradicional en un contexto de aceleración de los ciclos de negocio impulsado por internet, y analizamos en detalle el Manifiesto Ágil de 2001: sus cuatro valores, sus doce principios, y la idea central que los atraviesa —la entrega continua de valor real al cliente como criterio último de éxito de un proyecto—. Cerramos comparando la gestión del cambio en contextos ágiles y tradicionales, y estableciendo criterios concretos para decidir cuándo conviene un enfoque predictivo, un enfoque ágil o una combinación híbrida de ambos.

Estos conceptos no deben entenderse como compartimentos aislados: constituyen un cuerpo de conocimiento integrado que será la base sobre la cual, en los próximos módulos de esta materia, se profundizará en los marcos ágiles específicos —Scrum, Kanban, Extreme Programming— y en las técnicas concretas de planificación, estimación, seguimiento y mejora continua que un profesional de sistemas necesita dominar para liderar proyectos de software con éxito en el contexto actual de la industria.